Building a Service App for Armenia

https://gemini.google.com/app/3210060b8be5660e

User prompt: I need to create an app like with railway, like I'm in Armenia now so here to find electrician, plumber, driver, etc. so I need to create an app where clients who are looking for plumbers and other general workers (painters) all kind of workers online.

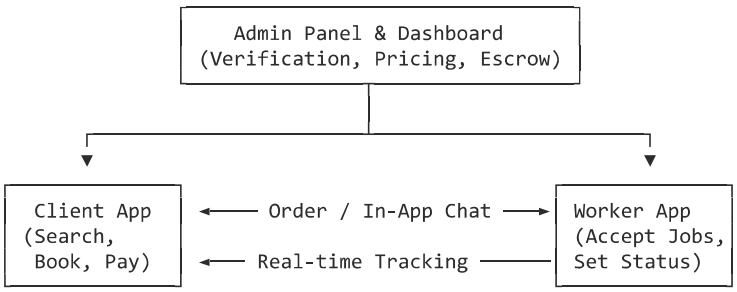
Response: Building an on-demand home and local services app tailored for Armenia is a promising idea, especially as digitizing daily blue-collar services (like *varpets*—craftsmen/handymen) continues to grow. Local users in Armenia often rely on **List.am** or word-of-mouth (Telegram/WhatsApp), but modern structured platforms like **Fixyan** or **ServiceNer** are showing that users prefer guaranteed pricing and verified professionals.

Here is a comprehensive breakdown of how to build, launch, and adapt this app for the Armenian market:

1. App Architecture & Key Features

Your platform will require **three core components**: a **Client App**, a **Worker (Pro) App**, and an **Admin Dashboard.**

Core Platform Features



- **Client Side:**
  - **Multi-language Support:** Essential for Armenia (Armenian, Russian, English).
  - **Booking Flow:** Up-front price quotes (flat-rate or hourly) or request-based bidding.
  - **In-app Chat & Calls:** Integrated audio calling or messaging to align on job details.
  - **Provider Ratings & Verification Badges:** Showing ratings, verified ID tags, and completed job counts.
- **Worker Side (*Varpet* Portal):**
  - **ID Verification Flow:** Passport/ID upload to get verified.
  - **Job Radius & Availability:** Map-based toggle to go online/offline and set work zones (e.g., Kentron, Arabkir, Gyumri).
  - **Earnings & Payout Tracker:** Clear summary of cash collected vs. commission owed to the platform.

2. Armenia-Specific Requirements

To succeed locally in Armenia, your tech stack and operations must align with local habits:

Area	Requirement / Adaptation
Payment Gateways	**Cash is King:** Allow "Cash on completion" as the primary default. For digital payments, integrate local providers: **Telcell Wallet**, **EasyPay**, **Idram**, and local bank card processing via **Ameriabank / ACBA vPOS.**
Legal & Compliance	Register a local LLC (e.g., via the State Register of Legal Entities) and ensure compliance with **Armenian Personal Data Protection laws.**
Localization	Default to Armenian (*Հայերեն*), but ensure full English and Russian translations to serve expats, tourists, and repatriates in Yerevan.
Communication	Integrations with **WhatsApp** or **Telegram API** as fallback channels, as local communication heavily relies on these apps.

3. Recommended Tech Stack

For a fast market entry with cross-platform capability:

- **Frontend (Mobile Apps): Flutter** or **React Native** (builds both iOS and Android from one codebase).
- **Backend & Database: Node.js / Go** with **PostgreSQL** (PostGIS enabled for geographic matching) or **Firebase** for rapid prototyping.

- **Maps & Location: Google Maps API** or **Mapbox** for geo-fencing regions in Yerevan and secondary cities like Gyumri and Vanadzor.
- **Push Notifications:** Firebase Cloud Messaging (FCM).

4. Execution Roadmap

1. **MVP Scope:** Focus first on **Yerevan** with 3–4 high-demand categories (Plumbing, Electrical, Handyman, Cleaning).
2. **Worker Onboarding Strategy:** Before launching to clients, recruit the first 50–100 workers through targeted social media campaigns (Facebook groups, Instagram) or direct outreach on local classifieds.
3. **Launch & Feedback:** Offer launch incentives (e.g., zero platform commission for the first month for workers) to build initial supply.

Would you like to explore **pricing models** (commission vs. subscription), look at **ready-made white-label software/templates** to speed up development, or discuss how to handle **local payment gateway integrations**?

Sources

1. On-demand home services in Yerevan | Cleaning, plumbing ... - Fixyan
2. Fixyan: Home Services Yerevan - Apps on Google Play
3. Find Services in Armenia | Trusted Provider Profiles - ServiceNer

---

User prompt: ok another thing is that I need the chats and calls directly through whatsapp and telegram as well is it possible ?

Response: Yes, **it is 100% possible**, and in fact, it is often preferred by local service workers in Armenia because most *varpets* (craftsmen), drivers, and plumbers already use WhatsApp and Telegram as their primary daily communication tools.

You have **two main ways** to implement this into your app:

Method 1: Deep Linking (Simplest & Direct)

Instead of building a complex messaging server inside your app, you can use **URL schemes (Deep Links)**. When a client taps "Chat on WhatsApp" or "Call on Telegram" inside your app, it automatically opens the native WhatsApp or Telegram app directly on the provider's contact page.

1. WhatsApp Integration

- **Direct Chat Link:** Format: `[https://wa.me/](https://wa.me/)<phone_number>?text=<prefilled_message>`
- **Example for Armenia (+374):** `[https://wa.me/37491123456?text=Hi,%20I%20found%20you%20on%20](https://wa.me/37491123456?text=Hi,%20I%20found%20you%20on%20)[YourApp]%20for%20a%20plumbing%20job.`
- **How it works:** Tapping this opens WhatsApp instantly with the phone number filled in and a pre-formatted introductory message ready to send.

2. Telegram Integration

- **Direct Chat Link:** Format: `[https://t.me/](https://t.me/)+<phone_number>` or `[https://t.me/](https://t.me/)<username>`
- **Example:** `[https://t.me/+37491123456](https://t.me/+37491123456)` or `[https://t.me/plumber_armenia](https://t.me/plumber_armenia)`
- **How it works:** Tapping this launches Telegram directly on the user's phone and opens a direct 1-on-1 chat or voice call window with the worker.

Method 2: Embedded Telegram Mini App (Advanced Option)

Instead of building a separate standalone mobile app from scratch, you can build your service as a **Telegram Mini App (Web App)**.

- **How it works:** Users open your Telegram Bot (e.g., `@ArmeniaFixBot`), which opens a rich mobile interface inside Telegram.
- **Why it works in Armenia:** Millions of users in Armenia already use Telegram daily. This eliminates the friction of forcing users or workers to download a new app from the App Store or Google Play Store. Clients can view, book, and chat with workers without ever leaving Telegram.

Important Considerations for Your Platform

PROS & CONS OVERVIEW	
Pros	Cons & Mitigations
• Zero learning curve • Higher response rate • Lower backend dev cost	• Platform Disintermediation (Users deal off-platform) • Loss of in-app call quality

```
                                    and conversation monitoring
```

1. **Platform Leakage (Disintermediation):**
    - *The Risk:* Once clients and workers connect on WhatsApp or Telegram, they may skip your platform for future jobs to avoid service fees.
    - *Solution:* Use a **lead-generation or model fee approach** (charge workers a small fee to receive client requests or list their profile, rather than taking a percentage per job).
2. **Privacy Settings:**
    - On Telegram, if a worker turns off *"Allow finding me by phone number,"* deep links by phone number will not open their profile. Workers will need to provide their Telegram **@username** during registration.

Suggested Elicitation

Sources

1. WhatsApp Deep Links: Complete 2026 Setup Guide - Visito's AI

User prompt: so first i need to create an automatic chat bot

Response: Building an automated chatbot is a practical first step to validate your local service network before investing in full native mobile apps. Starting with **Telegram** or **WhatsApp** lets you collect requests, route jobs, and test demand immediately.

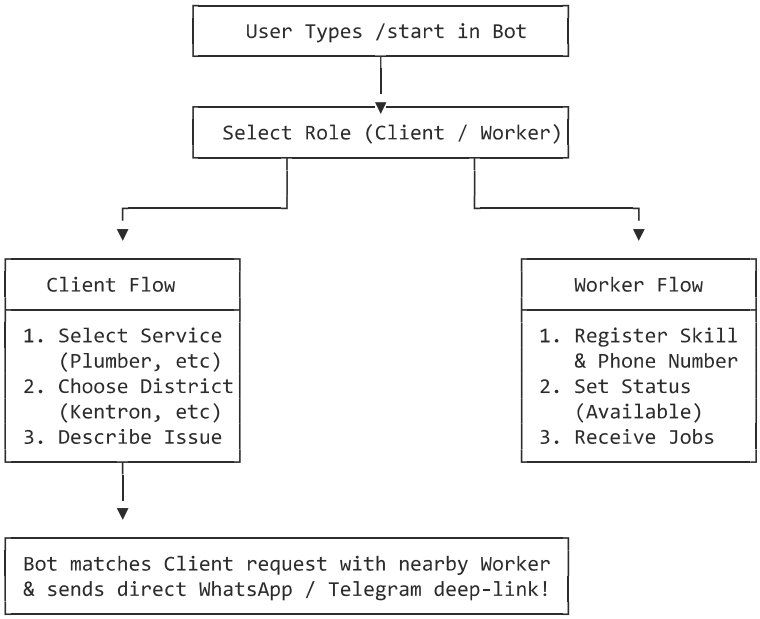
1. Choosing Your Bot Platform

Platform	Ease of Setup	Best Use Case	Recommended Tech
Telegram Bot	⚡ Very Easy (15 mins)	Ideal for quick prototyping, interactive inline buttons, and rich menus.	**Telegraf.js** / **python-telegram-bot**
WhatsApp Bot	⚙️ Moderate (Requires Meta API or Whapi)	Essential for older *varpets* (handymen) who exclusively use WhatsApp.	**WhatsApp Cloud API** or **Whapi.cloud**

Pro Tip: Start with **Telegram** to build your prototype fast, then plug the exact same conversation logic into **WhatsApp**.

2. Automated Chatbot Flow Architecture

Your bot should automatically guide users through options without needing human intervention:



3. Step-by-Step Telegram Setup (Quickstart)

Step 1: Register your bot with Telegram

1. Open Telegram and search for `@BotFather`.
2. Send `/newbot` and follow the prompts to choose a name and username.
3. Save the **HTTP API Token** provided by BotFather.

Step 2: Minimal Node.js Bot Code (`index.js`)

You can run this basic script on Node.js using the `telegraf` framework:

```
const { Telegraf, Markup } = require('telegraf');

// Replace with your token from @BotFather
const bot = new Telegraf('YOUR_TELEGRAM_BOT_TOKEN');

// 1. /start command
bot.start((ctx) => {
  ctx.reply('Welcome to Armenia Service Hub! AM\nHow can we help you today?',
    Markup.inlineKeyboard([
      [Markup.button.callback('🛠️ I need a Service (Client)', 'role_client')],
      [Markup.button.callback('🧰 I am a Worker / Varpet', 'role_worker')]
    ])
  );
});

// 2. Client Flow - Category Selection
bot.action('role_client', (ctx) => {
  ctx.reply('What service do you need?',
    Markup.inlineKeyboard([
      [Markup.button.callback('🚰 Plumber', 'cat_plumber'), Markup.button.callback('⚡ Electrician', 'cat_electrician')
      [Markup.button.callback('🚗 Driver', 'cat_driver'), Markup.button.callback('🎨 Painter / General', 'cat_painter')
    ])
  );
});

// 3. District Selection
bot.action(/cat_(.+)/, (ctx) => {
  const category = ctx.match[1];
  ctx.reply(`Selected: ${category.toUpperCase()}. Select your district in Yerevan:`,
    Markup.inlineKeyboard([
      [Markup.button.callback('Kentron', `dist_${category}_kentron`), Markup.button.callback('Arabkir', `dist_${category
      [Markup.button.callback('Avan', `dist_${category}_avan`), Markup.button.callback('Other', `dist_${category}_other`
    ])
  );
});

// 4. Connect to Worker via Direct WhatsApp / Telegram Link
bot.action(/dist_(.+)_(.+)/, (ctx) => {
  const category = ctx.match[1];
  const district = ctx.match[2];

  // Dummy worker phone number (In real production, pull from database)
  const workerPhone = '37491123456';
  const waLink = `https://wa.me/${workerPhone}?text=Hi!%20I%20found%20you%20on%20the%20ServiceBot%20for%20a%20${category

  ctx.reply(`We found available ${category}s in ${district}!`,
    Markup.inlineKeyboard([
      [Markup.button.url('💬 Chat with Worker on WhatsApp', waLink)]
    ])
  );
});

bot.launch();
console.log('Bot is running...');
```

4. No-Code Alternatives

If you prefer to build the bot visually without code first:

- **ManyChat:** Excellent for WhatsApp & Instagram automations.
- **Make.com / n8n:** Great for linking a Telegram bot to a **Google Sheet** or **Airtable** where workers' profiles are stored.

Would you like to build this using **Node.js code**, or would you prefer a **No-Code solution (n8n + Telegram + Google Sheets)** to get it live today?

Sources

1. How to create Telegram Chatbot with Node.js ? - GeeksforGeeks

---

User prompt: lets do it with node.js and i need it to be in two language, one in armenian and another in english so the people can choose which language they need. So start the coversation with selection of language. Then welcome message - "welcome to armworkforce armenia" then select the kind of worker you are looking for. and main thing before all that after the language selection he/she needs to provide an information that if he/she is looking for a worker or willing to work with us

Response: Here is a production-ready **Node.js Telegram bot** for **ArmWorkforce Armenia**.

It features **English & Armenian (Հայերեն) language switching**, role selection (Looking for a Worker vs. Willing to Work), service filtering, and deep-link generation for WhatsApp and Telegram.

1. Project Setup

Create a new directory on your computer or server and install the necessary dependencies:

```
mkdir armworkforce-bot
cd armworkforce-bot
npm init -y
npm install telegraf dotenv
```

Create a `.env` file in your root folder:

```
BOT_TOKEN=YOUR_TELEGRAM_BOT_TOKEN_FROM_BOTFATHER
```

2. Complete Code (`index.js`)

Create a file named `index.js` and paste the following code:

```
require('dotenv').config();
const { Telegraf, Markup, session } = require('telegraf');

if (!process.env.BOT_TOKEN) {
  console.error("Error: BOT_TOKEN is missing in .env file.");
  process.exit(1);
}

const bot = new Telegraf(process.env.BOT_TOKEN);

// Enable session storage to track language & user state
bot.use(session());

// Initialize default session data
bot.use((ctx, next) => {
  if (!ctx.session) {
    ctx.session = { lang: 'en', role: null };
  }
  return next();
});

// Translations Dictionary
const i18n = {
  en: {
    welcome: "Welcome to **ArmWorkforce Armenia**! AM\n\nPlease select your intent below:",
    opt_looking: "🔍 Looking for a Worker",
    opt_worker: "🛠️ Willing to Work with Us",
    select_service: "Please select the category of worker you are looking for:",
    worker_registration: "Thank you for joining ArmWorkforce! AM\nPlease send us your full name, phone number, and skill
    categories: {
      plumber: "🚰 Plumber",
      electrician: "⚡ Electrician",
      driver: "🚗 Driver",
      painter: "🎨 Painter / General Worker",
      cleaner: "🧹 Cleaner / Housekeeping"
    },
    results: (category) => `We found available professionals for **${category}** in Yerevan!\nClick below to connect dir
    btn_whatsapp: "💬 Chat on WhatsApp",
    btn_telegram: "✈️ Chat on Telegram",
    btn_back: "◀️ Back"
```

```
  },
  hy: {
    welcome: "Բարի գալուստ **ArmWorkforce Armenia**! Ամ\n\nԽնդրում ենք ընտրել տարբերակը.",
    opt_looking: "🔍 Փնտրում եմ աշխատող",
    opt_worker: "🛠️ Ցանկանում եմ աշխատել մեզ հետ",
    select_service: "Խնդրում ենք ընտրել աշխատողի տեսակը, որին փնտրում եք.",
    worker_registration: "Շնորհակալություն ArmWorkforce-ին միանալու համար: Ամ\nԽնդրում ենք ուղարկել Ձեր անունը, հեռախոսա
    categories: {
      plumber: "🚰 Ջրմուղագործ (Plumber)",
      electrician: "⚡ Էլեկտրիկ (Electrician)",
      driver: "🚚 Վարորդ (Driver)",
      painter: "🎨 Ներկարար / Համարյա բոլոր գործեր",
      cleaner: "🧹 Մաքրուհի / Մաքրություն"
    },
    results: (category) => `Գտնվել են հասանելի մասնագետներ **${category}** կատեգորիայում:\nՍեղմեք ստորև անմիջապես կապ հա
    btn_whatsapp: "💬 Կապվել WhatsApp-ով",
    btn_telegram: "✈️ Կապվել Telegram-ով",
    btn_back: "◀️ Ուղղվել ետ"
  }
};

// ------------------------------------------------------------
// 1. /start command - Step 1: Language Selection
// ------------------------------------------------------------
bot.start((ctx) => {
  ctx.session = { lang: 'en', role: null }; // Reset session on start

  ctx.reply(
    "Please select your language / Խնդրում ենք ընտրել լեզուն.",
    Markup.inlineKeyboard([
      [
        Markup.button.callback('🇬🇧 English', 'lang_en'),
        Markup.button.callback('🇦🇲 Հայերեն', 'lang_hy')
      ]
    ])
  );
});

// Helper function to get text based on user session
const t = (ctx) => i18n[ctx.session.lang || 'en'];

// ------------------------------------------------------------
// 2. Language Selection Callbacks - Step 2: Welcome & Role Selection
// ------------------------------------------------------------
bot.action(/lang_(en|hy)/, async (ctx) => {
  await ctx.answerCbQuery();
  ctx.session.lang = ctx.match[1];
  const txt = t(ctx);

  await ctx.editMessageText(
    txt.welcome,
    {
      parse_mode: 'Markdown',
      ...Markup.inlineKeyboard([
        [Markup.button.callback(txt.opt_looking, 'role_client')],
        [Markup.button.callback(txt.opt_worker, 'role_worker')]
      ])
    }
  );
});

// ------------------------------------------------------------
// 3. Role Selection Callbacks
// ------------------------------------------------------------

// Option A: Willing to work with us (Worker Registration)
bot.action('role_worker', async (ctx) => {
  await ctx.answerCbQuery();
  ctx.session.role = 'worker';
  const txt = t(ctx);

  await ctx.editMessageText(
```

```
      txt.worker_registration,
      Markup.inlineKeyboard([
        [Markup.button.callback(txt.btn_back, 'back_to_roles')]
      ])
    );
  });

  // Option B: Looking for a worker (Client Flow) -> Step 3: Select Service Category
  bot.action('role_client', async (ctx) => {
    await ctx.answerCbQuery();
    ctx.session.role = 'client';
    const txt = t(ctx);

    await ctx.editMessageText(
      txt.select_service,
      Markup.inlineKeyboard([
        [Markup.button.callback(txt.categories.plumber, 'cat_plumber')],
        [Markup.button.callback(txt.categories.electrician, 'cat_electrician')],
        [Markup.button.callback(txt.categories.driver, 'cat_driver')],
        [Markup.button.callback(txt.categories.painter, 'cat_painter')],
        [Markup.button.callback(txt.categories.cleaner, 'cat_cleaner')],
        [Markup.button.callback(txt.btn_back, 'back_to_roles')]
      ])
    );
  });

  // ------------------------------------------------------------
  // 4. Back Button Handler
  // ------------------------------------------------------------
  bot.action('back_to_roles', async (ctx) => {
    await ctx.answerCbQuery();
    const txt = t(ctx);

    await ctx.editMessageText(
      txt.welcome,
      {
        parse_mode: 'Markdown',
        ...Markup.inlineKeyboard([
          [Markup.button.callback(txt.opt_looking, 'role_client')],
          [Markup.button.callback(txt.opt_worker, 'role_worker')]
        ])
      }
    );
  });

  // ------------------------------------------------------------
  // 5. Category Selection & Direct WhatsApp / Telegram Deep-Linking
  // ------------------------------------------------------------
  bot.action(/cat_(.+)/, async (ctx) => {
    await ctx.answerCbQuery();
    const categoryKey = ctx.match[1];
    const txt = t(ctx);
    const categoryLabel = txt.categories[categoryKey] || categoryKey;

    // Example representative/worker contact info (Armenia format +374)
    const contactPhone = '37491123456';
    const contactTelegramUser = 'armworkforce_support';

    // Pre-filled WhatsApp message
    const prefilledText = encodeURIComponent(`Hi, I found your service on ArmWorkforce for a ${categoryKey} job.`);
    const whatsappUrl = `https://wa.me/${contactPhone}?text=${prefilledText}`;
    const telegramUrl = `https://t.me/${contactTelegramUser}`;

    await ctx.editMessageText(
      txt.results(categoryLabel),
      {
        parse_mode: 'Markdown',
        ...Markup.inlineKeyboard([
          [Markup.button.url(txt.btn_whatsapp, whatsappUrl)],
          [Markup.button.url(txt.btn_telegram, telegramUrl)],
          [Markup.button.callback(txt.btn_back, 'role_client')]
        ])
      }
    );
```

```
});

// Launch Bot
bot.launch();
console.log('🚀 ArmWorkforce Bot is successfully running...');

// Enable graceful stop
process.once('SIGINT', () => bot.stop('SIGINT'));
process.once('SIGTERM', () => bot.stop('SIGTERM'));
```

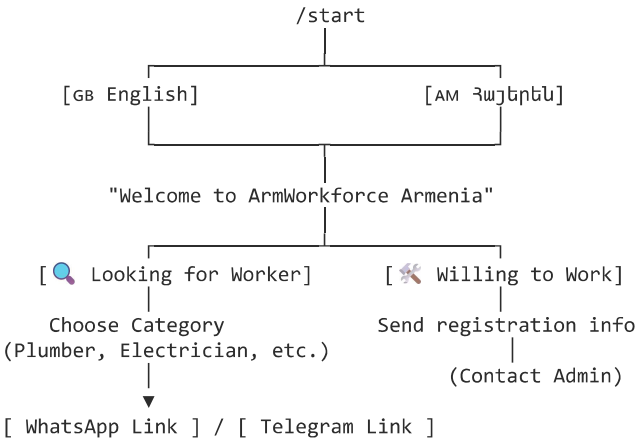
3. How to Run & Test

1. Get a token from **@BotFather** on Telegram.
2. Put the token inside your `.env` file.
3. Run the application:

   ```
   node index.js
   ```

4. Open your bot on Telegram and type `/start`.

Conversation Flow Structure



Would you like to connect this bot to a **database** (like PostgreSQL or Firebase) to automatically save registered workers, or integrate an administrative dashboard to manage requests?

User prompt: how about linking it with railway ?

Response: Deploying your Node.js Telegram bot on **Railway** is a great choice. Railway handles hosting, keeps your bot running 24/7 in the cloud, and links seamlessly with GitHub.

Because Telegram bots use **long polling** by default (meaning the bot constantly checks Telegram for new messages), it will run continuously on Railway without needing complex webhook setups.

Here is how to deploy your bot step-by-step:

Step 1: Prepare Your Project Files for Railway

Make sure your project folder has these three essential files:

1. `package.json`

Ensure your `start` script is properly defined so Railway knows how to run your application. Inside your `package.json`, check that the `scripts` section looks like this:

```
"scripts": {
  "start": "node index.js"
}
```

2. `.gitignore`

Always include a `.gitignore` file so you don't accidentally upload your local environment tokens to GitHub:

```
node_modules/
.env
```

3. `index.js`

Your bot code (from the previous step) using `telegraf` and `dotenv`.

Step 2: Push Your Code to GitHub

1. Create a new repository on your GitHub account (e.g., `armworkforce-bot`).
2. Open your terminal in your project folder and run:

```
git init
git add .
git commit -m "Initial bot commit"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_USERNAME/armworkforce-bot.git
git push -u origin main
```

Step 3: Deploy on Railway

1. Go to Railway.app and log in using your GitHub account.
2. Click **Start a New Project** -> **Deploy from GitHub repo**.
3. Select your `armworkforce-bot` repository from your list.
4. Railway will automatically detect that it is a Node.js project and begin building it using Nixpacks.

Step 4: Add Your Environment Variables

Before the build finishes or right after, you need to provide your Telegram Bot Token safely:

1. Click on your deployed service tile inside your Railway project dashboard.
2. Go to the **Variables** tab.
3. Click **New Variable** and add:
   - **Key:** `BOT_TOKEN`
   - **Value:** `YOUR_TELEGRAM_BOT_TOKEN` (the token you got from `@BotFather`)
4. Click **Add**. Railway will automatically trigger a re-build with your token securely injected.

Step 5: Test Your Live Bot

1. Check the **Deployments / Logs** tab in Railway to make sure you see:

   🚀 `ArmWorkforce Bot is successfully running...`

2. Open Telegram, search for your bot handle, and type `/start`. It will instantly respond with the Armenian/English language selector!

Suggested Elicitation

Sources

1. Deploy & Host Telegram Bot (Telegraf) - Railway
2. Deploy & Host Telegram Bot - Railway
3. Deploying a Telegram Bot using ChatGPT and Whisper APIs with
4. Railway - BuilderBot.app Chatbot for Whatsapp, Telegram and more

---